

Week 1: Deep Latent Variable Models (DLVMs)

Generative Models: The decoder $f_\theta : \mathcal{Z} \rightarrow \mathcal{H}$ transforms latent representations into parameters of the output density $\eta = f_\theta(\mathbf{z})$. Output densities depend on data type (e.g., Categorical for images, Gaussian for continuous data).

VAEs & ELBO: Models are learned via amortized variational inference by optimizing the Evidence Lower Bound:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - KL(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$$

Priors & Posterior Collapse: The standard Gaussian prior can cause posterior collapse. Better priors include Mixture of Gaussians (MoG) or VampPrior, a mixture of approximate posteriors:

$$p_\lambda(\mathbf{z}) = \frac{1}{K} \sum_{k=1}^K q_\phi(\mathbf{z}|\mathbf{u}_k)$$

Shared decoders in hierarchical VAEs also help prevent posterior collapse.

Week 2: Normalizing Flows

Concept: A sequence of invertible, differentiable transformations (diffeomorphisms) maps from a base distribution such as $\mathcal{N}(0, \mathbf{I})$:

$$\mathbf{x} = T(\mathbf{u}) = T_K \circ \dots \circ T_1(\mathbf{u})$$

Exact Likelihood: Density is computed using the change of variables formula:

$$p_x(\mathbf{x}) = p_u(\mathbf{u}) |\det J_T(\mathbf{u})|^{-1}$$

Dequantization: To apply continuous flows to discrete data, uniform noise is added: $\mathbf{x}' = \mathbf{x} + \mathbf{u}$, where $\mathbf{u} \sim \mathcal{U}(-0.5, 0.5)$.

Affine Coupling (RealNVP): Partitions the input to apply identity and affine transforms:

$$\mathbf{z}'_{1:d} = \mathbf{z}_{1:d}, \quad \mathbf{z}'_{d+1:D} = \mathbf{z}_{d+1:D} \odot \exp(s) + t$$

The Jacobian is triangular, so $\det J = \exp(\sum s)$. RealNVP uses checkerboard and channel masking.

Glow: Extends RealNVP by incorporating invertible 1×1 convolutions as a generalization of permutation layers. Flows can also serve as highly expressive priors for VAEs.

Week 3: Diffusion Models

Forward Process: A Markov chain gradually corrupts data into noise:

$$q(\mathbf{z}_t|\mathbf{z}_{t-1}) = \mathcal{N}(\mathbf{z}_t; \sqrt{1 - \beta_t} \mathbf{z}_{t-1}, \beta_t \mathbf{I})$$

The marginal is $q(\mathbf{z}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{z}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$.

Reverse Process: A neural network (often a U-Net) approximates the denoising step:

$$p(\mathbf{z}_{t-1}|\mathbf{z}_t) = \mathcal{N}(\mathbf{z}_{t-1}; \boldsymbol{\mu}_\theta, \boldsymbol{\Sigma}_\theta)$$

DDPM Loss: The ELBO simplifies to predicting the noise $\boldsymbol{\epsilon}$ via MSE:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\boldsymbol{\epsilon}, t} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{z}_t(\mathbf{x}_0, \boldsymbol{\epsilon}), t)\|^2]$$

Continuous SDEs: Modeled as $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{w}$. Reversing requires the score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, learned via denoising score matching. Variance Preserving (VP) SDEs use a linear noise schedule $\beta(t)$.

Latent Diffusion: Computations scale better by first projecting data into a low-dimensional VAE latent space before running the diffusion process.

Week 5: Representation Invariance

Identifiability: Too much flexibility in neural network decoders means latent representations can be reparameterized without changing the model fit, leading to unstable or unidentifiable representations.

Noisy Manifolds: Observations are modeled as an immersed manifold plus noise:

$$\mathbf{x} = \mathbf{W}f(\mathbf{z}) + \boldsymbol{\epsilon}$$

where $\mathbf{W}$ is interpreted as a random projection.

Ensembles: Capturing model uncertainty (e.g., using multiple randomly initialized decoders) is vital for understanding topology. Well-calibrated uncertainty builds a “wall” around the data so that geodesic curves naturally follow the data’s topological structure.

Week 6: Riemannian Metrics

Riemannian Geometry: A deterministic decoder $f : \mathbb{R}^d \rightarrow \mathbb{R}^D$ immerses a manifold in observation space. Locally, distances and angles in the tangent space are defined by the pull-back Riemannian metric:

$$\mathbf{M}_z = J_f(\mathbf{z})^\top J_f(\mathbf{z})$$

Curve Energy & Geodesics: The energy of a curve γ is:

$$E[\gamma] = \int_0^1 \gamma'(t)^\top \mathbf{M}_{\gamma(t)} \gamma'(t) dt$$

Shortest paths (geodesics) minimize this energy.

Optimization: Constant-speed geodesics follow a 2nd-order ODE, providing a principled way to generalize addition and subtraction on the learned manifold.

Week 7: Information Geometry & Parameter Manifolds

Information Geometry: Treats the decoder as mapping to the space of conditional distributions $p(\mathbf{x}|\mathbf{z})$ rather than directly to observation space. Distances are measured using the KL divergence, which is invariant to parameterization choices.

Parameter Manifolds: The framework extends to any neural network parameter. Network weights can be treated as latent variables; their geometry (metric) is evaluated by measuring the change in network predictions on the training data. Sampling in parameter space using this geometry allows samples to interpolate the data perfectly.

Week 9: Graphs & Node Embeddings

Graph Basics: A simple graph is $G = (V, E)$. Laplacians capture structural properties:

$$\text{Unnormalized: } \mathbf{L} = \mathbf{D} - \mathbf{A}, \quad \text{Random Walk: } \mathbf{L}_{rw} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$$

The Weisfeiler-Lehman test checks graph isomorphism by iteratively hashing the multi-set of neighborhood labels.

Node Embeddings: The encoder maps nodes to vectors (e.g., shallow embedding $\mathbf{z}_u = \mathbf{s}_u^\top \mathbf{Z}$). The decoder computes a node-pair statistic such as dot product $\mathbf{z}_u^\top \mathbf{z}_v$, squared distance, or sigmoid probability.

Losses: Optimization minimizes discrepancies like MSE ($\|\mathbf{S} - \mathbf{Z}\mathbf{Z}^\top\|_F^2$), binary cross-entropy, or uses random walks with negative sampling.

Week 10: Graph Neural Networks

Message Passing: State updates are order-invariant:

$$\mathbf{h}_u^{(k+1)} = \text{update}(\mathbf{h}_u^{(k)}, \text{agg}(\{\mathbf{h}_v^{(k)} \mid v \in \mathcal{N}(u)\}))$$

Aggregation: Must be permutation invariant (e.g., sum, mean). Common approaches include set pooling, Janossy pooling (averaging over randomized permutations), and Transformer-style attention:

$$\alpha_{u,v} \propto \exp(\mathbf{h}_u^\top \mathbf{W} \mathbf{h}_v)$$

Updates: Concatenation, residual skip-connections, or GRU-based gating help combat over-smoothing in deep networks.

Geometric GNNs: Physical symmetries (translation and rotation) are handled via invariant/equivariant operations on separate scalar and vector features.

Week 11: Graph Theory & Generative Graph Models

Graph Convolutions: A graph convolution uses the adjacency matrix $\mathbf{A}$ as a neighborhood shift operator. Via the eigendecomposition $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^H$, the eigenvectors correspond to the Discrete Fourier Transform basis, enabling spectral convolutions.

Graphical Models: Mean-field variational inference fixed-point equations are analytically similar to GNN message passing updates.

Generative Graph Models: Classical approaches include Erdős-Rényi, Stochastic Block-models, and Preferential Attachment. VAEs for graphs use a probabilistic encoder:

$$\mathbf{Z} \sim \mathcal{N}(\mu(G), \Sigma(G))$$

to build node-level or graph-level latent representations. Performance is evaluated by comparing distributions of graph statistics (degree, clustering coefficient) using the Total Variation (TV) distance.